Custom Reinforcement Learning Agent Using Gymnasium

Topics in AI | Assignment I | COMSATS University Islamabad, Attock Campus

1. Problem Description

The task is to learn a policy for the 8x8 slippery FrozenLake-v1 benchmark. An agent moves from a start tile to a goal while avoiding holes. Slipperiness makes the chosen movement stochastic, so the agent must learn a robust path. The discrete observation is the integer tile index (0–63). The four actions are left (0), down (1), right (2), and up (3). Reaching the goal yields +1; all other transitions yield 0. Falling in a hole or reaching the goal terminates the episode; the configured time limit is 100 steps.

2. Methodology and Architecture

The implementation uses off-policy tabular Q-learning with epsilon-greedy exploration. A 64x4 NumPy array stores action values, initialized to zero. At each transition, the algorithm updates the selected action value:

Q(s,a) = Q(s,a) + alpha [r + gamma max(Q(s′, a′)) - Q(s,a)]. For a terminal transition, the bootstrap term is set to zero. Time-limit truncation is not treated as a terminal state for the target. Exploration uses epsilon, multiplied by a decay factor after each episode and bounded below by epsilon_min. The action selection in evaluation is greedy.

Architecture: **agent.py** contains the Q-learning agent; **main.py** handles environment creation, training, evaluation, and sweeps; **utils.py** generates plots and a policy GIF; **experiment.ipynb** presents reproducible results. Gymnasium provides only the environment interface; the algorithm is implemented from scratch.

3. Experimental Setup and Hyperparameters

Parameter	Final setting
Environment	FrozenLake-v1, 8x8, is_slippery=True
Learning rate (alpha)	0.10
Discount factor (gamma)	0.99
Initial / minimum epsilon	1.0 / 0.05
Epsilon decay	0.9997 per episode
Training episodes	30,000
Episode maximum length	100 steps
Evaluation	500 independent greedy episodes
Random seed	42 (evaluation starts at 100000)

4. Results and Discussion

Reproducibility status: The code and experiment protocol are provided, but the Gymnasium dependency could not be installed in the current execution environment. Therefore, no measured training results, plot images, trained Q-table, or policy GIF are claimed in this report. Run **python main.py** in a Python environment with dependencies installed to generate those artifacts and populate the fields below.

Report the following from **outputs/results.json**: final greedy evaluation success rate = **[INSERT]**; average episode length = **[INSERT]**; last-1000 training success rate = **[INSERT]**; successful GIF recorded = **[INSERT]**. Insert the generated **training.png** and **sensitivity.png** plots here before final submission. The sensitivity sweep tests (alpha, gamma) pairs (0.05,0.95), (0.10,0.95), (0.10,0.99), and (0.20,0.99), holding the training budget fixed.

Interpretation guide: convergence is assessed by the 200-episode moving-average reward curve, while the final greedy success rate measures out-of-sample performance. Compare the four evaluation success rates in sensitivity.csv.

Slippery dynamics can cause failure even under a learned policy; sparse goal-only rewards, exploration decay, and random seeds can delay convergence. A single run is insufficient to establish statistical superiority; multiple independent training seeds would improve reliability.

5. Conclusion

This assignment provides a reproducible implementation of tabular Q-learning for stochastic navigation. The training, evaluation, visualization, and sensitivity pipeline is included. Empirical conclusions must be finalized after running the supplied code, verifying the saved policy GIF, and inserting the resulting figures and statistics.

References

Farama Foundation. Gymnasium documentation: https://gymnasium.farama.org/ . Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. Machine Learning, 8, 279–292.